

RAG Pipeline: A Practical Guide

Introduction to Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines information retrieval with a large language model. A RAG application retrieves relevant passages from a document collection and provides those passages to the language model as context before generating an answer.

Document Processing

Documents such as PDF files can be divided into smaller text chunks. Chunking makes retrieval more precise because the vector database can return focused sections instead of an entire document. Chunk size and overlap are important because very small chunks may lose context while very large chunks may contain unrelated information.

Embeddings and ChromaDB

An embedding model converts text into numerical vectors. Texts with similar meanings tend to have vectors that are close together. ChromaDB is a vector database that can store these embeddings along with the original text and metadata such as the source filename, page number, and chunk identifier.

LangChain and Groq

LangChain connects document loaders, text splitters, embedding models, vector stores, retrievers, prompts, and language models. Groq provides access to language models through an API. In this project, the Groq model receives the user's question together with retrieved document context and generates the final answer.

Source Citations

Source citations make a RAG chatbot more trustworthy. Retrieved chunks should retain metadata showing where they came from. The final response can display the document name and page or chunk information so users can verify the answer.

Testing Questions

1. What is RAG? 2. Why is document chunking used? 3. What does ChromaDB store? 4. What is the role of LangChain? 5. What is Groq used for? 6. Why are source citations important?